```
$ bundle config --delete bin
$ rails app:update:bin
```

When you generate a Rails object such as a controller or scaffold, RSpec will create a corresponding spec file for you:

```
$ rails generate model pterodactyl
      invoke  active_record
      create    db/migrate/20170613203526_create_pterodactyls.rb
      create    app/models/pterodactyl.rb
      invoke    rspec
      create      spec/models/pterodactyl_spec.rb
```

You can also generate just the spec file, if the class you're testing already exists or if you don't want to create it just yet; just prepend the item you're generating with rspec:, as in rails generate rspec:model pterodactyl.

Next, let's take a look at the different types of specs you might write to test your Rails app.

Spec Types

For the expense tracker API you built in this book, you wrote three different kinds of specs:

- Acceptance specs to test the entire app end to end
- Unit specs to test one layer in isolation
- Integration specs to test objects with real collaborators and external services

A Rails app is more complex than a tiny API. Accordingly, Rails provides infrastructure for several different kinds of tests, including the following ones:

- Integration tests that drive your app as a black box via its HTTP interface
- Functional tests to see how your controllers respond to requests
- Unit tests to drive a single object or layer
- Specific tests for models, mailers, and background jobs; any given test here may be a unit or integration test

To test one of these aspects of your app in RSpec, tag your example group with the :type metadata, passing it one of the spec types from the chart in the next section (:model, :request, :helper, and so on). For example, here's the spec file generated by the rails generate model pterodactyl command from the previous section: